Name :MONISHAVRAN
REG NO: 192424398

DSA0310 – Natural Language Processing

Assignment 1 (CO1): Model Answers – Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System

Note: Measured values (counts, precision, recall, F1, run-times, type-to-lemma ratio) depend on your own corpus and code. They are marked **[from your run]**. Everything else (concepts, rules, worked examples and design decisions) is given here in full.

Task 1 – Frame the Problem (3 marks)

Levels of linguistic analysis

Level	Role in the record-search problem	Corpus example
Phonology	Ignore (text only). Only matters for misspellings that sound alike.	"nephrectomy" typed as "nefrectomy"
Morphology	Core requirement: relates fractures, fractured, fracturing to fracture.	fractures → fracture +N +PL
Syntax	Not needed for keyword search.	Word order in "patient denies chest pain"
Semantics	Slightly needed: renal ≈ kidney (lexical relatedness).	nephrectomy / renal
Pragmatics	Ignore for search.	Intent behind "no fracture seen"
Discourse	Ignore, except negation scope in advanced systems.	Pronoun "it" referring to a lesion

Three model families

Family	Best hospital task	Why the other two are wrong
State machines (FSA/FST)	Validating patient IDs; morphological parsing with an FST	Rules alone are harder to run in both directions; probabilistic models waste training data on a fixed format
Formal rule systems	Orthographic rules (e-deletion) and Porter's rewrite rules	A state machine is awkward for context-sensitive spelling rewrites; statistics would be unreliable for rare terms
Probabilistic models	Choosing among ambiguous analyses (leaves); language ID	Hand rules cannot weigh competing evidence; a plain automaton only accepts or rejects

Ambiguity

Ambiguity is the central problem because the same surface string maps to several interpretations, and every later stage depends on picking the right one. Examples:

- **Lexical:** "discharge" is a noun (hospital discharge, wound discharge) or a verb.
- **Morphological:** "leaves" is leaf +N +PL or leave +V +3SG.
- **Segmentation:** "5mg" vs "5 mg"; "b.i.d." with full stops may be split wrongly at the dots.

- **Abbreviation:** "MS" is multiple sclerosis, mitral stenosis or morphine sulphate.

Where search appears and what it costs

- **State-space search:** NFA simulation tracks the set of live states; the worst case is 2^n DFA states after subset construction.
- **Backtracking:** regex engines try one alternative and return to a choice point on failure; the worst case is exponential time.
- **Resources consumed:** time and memory for the machines; annotated data for evaluation; expert effort for lexicon and rule writing.

Pipeline

#	Stage	Input	Output	Formalism
1	Tokeniser	Raw text	Token stream	Regular expression (maximal munch)
2	Entity recogniser	Tokens/text	Typed entities (ID, date, dose…)	Regex + DFA validators
3	Normaliser	Tokens	Case-folded, abbreviations expanded, canonical dates/units	Regex substitutions
4	Morphological analyser	Normalised words	Stem + features	FST with continuation classes + orthographic rule cascade
5	Fallback stemmer	Out-of-lexicon words	Stem	Porter rule system
6	Indexer / search	Stems and entities	Ranked records	Inverted index

Task 2 – Basic Regular Expression Patterns (4 marks)

Basic operators with clinical examples

Operator	Syntax	Minimal clinical example
Concatenation	ab	mg matches the unit mg
Disjunction	a|b	mg|mcg|ml matches a unit
Kleene star	a*	\d* matches zero or more digits
Kleene plus	a+	\d+ matches a dose number
Optionality	a?	\d+\.?\d* matches 5 or 5.5
Character class	[abc]	[Mm][Gg] matches mg, MG
Range	[0-9]	[A-Z] matches ICD letter
Negated class	[^0-9]	[^0-9]+ matches a non-numeric run
Anchors	^ $ \b	\bBD\b matches BD as a whole word
Counters	{n,m}	\d{10} matches a phone number

Pattern inventory (12 entity types)

Entity	Regex	Must match	Must NOT match
Patient ID	\bPT-\d{6}\b	PT-102345	PT-12345 (5 digits)
Admission/discharge date	\b\d{1,2}[/-]\d{1,2}[/-](\d{4}|\d{2})\b	12/03/2024	12/03 (no year)
Drug + strength	\b[A-Z][a-z]+\s\d+(\.\d+)?\s?(mg|mcg|g|ml)\b	Paracetamol 500mg	Paracetamol alone
Dosage frequency	\b(OD|BD|TDS|QID|SOS|b\.?i\.?d\.?)\b	BD, b.i.d.	BDS (not a frequency)
Lab value + unit	\b\d+(\.\d+)?\s?(mg/dL|mmol/L|g/dL|%)	Hb 11.5 g/dL	11.5 alone
Blood pressure	\b\d{2,3}/\d{2,3}\s?mmHg\b	120/80 mmHg	12/03/2024
Temperature	\b\d{2,3}(\.\d)?\s?°?[CF]\b	38.5°C	38.5 kg
ICD code	\b[A-TV-Z]\d{2}(\.\d{1,4})?\b	S72.0	S7 (too short)
Ward and bed	\bWard\s?[A-Z0-9]+,?\s?Bed\s?\d{1,3}\b	Ward B, Bed 12	Bed 12 alone
Contact number	\b(\+91[-]?)?[6-9]\d{9}\b	+91 9876543210	0123456789
Age	\b\d{1,3}\s?(yrs?|years?|y/o)\b	45 yrs	45 mg
Symptom duration	\b\d+\s?(days?|weeks?|months?|hrs?)\b	3 days	3 daysx

Greedy vs non-greedy

Sentence: "give 5mg then 10mg daily".

```
greedy     5.*mg   ->  '5mg then 10mg'
non-greedy 5.*?mg  ->  '5mg'
```

The greedy `*` first consumes everything to the end of the string, then backtracks one character at a time until `mg` can match, so it stops at the last mg. The non-greedy `*?` begins with zero characters and expands one at a time, so it stops at the first mg.

Precedence and grouping

Precedence from highest to lowest: parentheses, counters/star/plus/optional, concatenation, disjunction. A misplaced disjunction therefore changes the language:

```
gray|grey day    denotes {gray, grey day}
gr(a|e)y day     denotes {gray day, grey day}
```

Evaluation and error causes

Error class	Linguistic cause
False positive	Number-like strings: a room number read as an age, or 12/03 read as a blood pressure
False negative	Spacing and format variants (5 mg / 5mg / 5 MG), spelling variation, OCR noise
True positive / FP / FN counts	[from your run]: tabulate per pattern

Task 3 – Regular Expressions for Tokenisation and Normalisation (4 marks)

Tokenizer

```
(?:[A-Za-z]\.){2,}|\d+(?:\.\d+)?|\w+(?:-\w+)*|[^\w\s]
```

- **Abbreviations with full stops** (b.i.d.) come first so they stay one token.
- **Decimals** (2.5) stay one token instead of 2 / . / 5.
- **Hyphenated terms** (anti-hypertensive) stay one token because they are one lexical unit.
- **Units against numbers:** 5mg splits into 5 and mg, so units can be normalised separately.

Python's alternation is ordered, not longest-match, so the alternatives are arranged with the most specific first to achieve the maximal-munch result.

Difference from whitespace splitting

Input	Whitespace split	Regex tokenizer
Take 2.5mg b.i.d.	[Take][2.5mg][b.i.d.]	[Take][2.5][mg][b.i.d.]
anti-hypertensive,	[anti-hypertensive,]	[anti-hypertensive][,]
BP 120/80mmHg.	[BP][120/80mmHg.]	[BP][120][/][80][mmHg][.]

Normalisation rules

Case folding: lower-case all tokens except those in an entity whitelist. Expand at least 30 abbreviations, for example:

Abbrev.	Expansion	Abbrev.	Expansion
BD	twice daily	HTN	hypertension
TDS	three times daily	DM	diabetes mellitus
OD	once daily	c/o	complains of
SOS	as needed	h/o	history of
pt	patient	Rx	prescription
NPO	nothing by mouth	Hb	haemoglobin
BP	blood pressure	SOB	shortness of breath
#	fracture	IVF	intravenous fluids
Tab	tablet	Inj	injection
PRN	as needed	Adm	admission

Extend the table to 30 or more entries. Dates are canonicalised to ISO (12/03/24 → 2024-03-12) and units to a single spelling (MG, Mg → mg). Show one full record before and after **[from your corpus]**.

Irreversible normalisation

- **Case folding:** US vs us; the original capitalisation is lost.
- **Abbreviation expansion:** an ambiguous abbreviation such as MS is expanded to one reading and the other readings are lost.
- **Unit/date canonicalisation:** the original written format (useful for OCR audit) is lost.

The loss is acceptable for search and indexing, because the original text is stored alongside. It is not acceptable for legal records, for audit, or when an ambiguous abbreviation is resolved wrongly in the stored copy.

Limit of regularity

Matching balanced parentheses in nested notes, e.g. "(history (of (MI)))". A finite automaton has a fixed number of states and cannot count unbounded nesting depth. This is proved formally in Task 4.

Task 4 – Finite-State Automata (5 marks)

Formal definition

A finite automaton is a 5-tuple (Q, Σ, δ, q0, F): Q is a finite set of states, Σ the input alphabet, δ the transition function (Q × Σ → Q for a DFA; Q × (Σ ∪ {ε}) → 2^Q for an NFA), q0 ∈ Q the start state, and F ⊆ Q the accepting states.

Four DFA validators (≥ 6 states each)

Draw the transition diagram and table for each. Example, **Patient ID PT-dddddd** (states q0–q8):

State	P	T	-	digit (0-9)
q0 (start)	q1	–	–	–
q1	–	q2	–	–
q2	–	–	q3	–
q3	–	–	–	q4
q4 … q8	–	–	–	next state
q9 (accept)	–	–	–	–

The other three validators: **date dd/mm/yyyy**, **blood pressure ddd/ddd**, **ICD code Ldd.d**. Each is built the same way with a dead (trap) state for all missing transitions.

NFA with ε-transitions: phone number with optional country code

```
Language: (\+91)?[6-9]\d{9}
```

States: q0 –ε→ q1 (skip prefix) and q0 –+→ q1a –9→ q1b –1→ q1 (prefix), then q1 –[6-9]→ q2 –digit→ … q11 (accept, after 9 more digits).

Accepted string +919876543210: live sets {q0,q1} → {q1a} → {q1b} → {q1} → {q2} → … → {q11} accept.

Rejected string 12345: {q0,q1} → no move on digit 1 (not 6-9, not +) → {} (empty set), reject.

Subset construction (outline)

DFA state	On +	On [6-9]	On other digit
A = ε-closure(q0) = {q0,q1}	B = {q1a}	C = {q2}	∅
B = {q1a}	–	–	– (needs 9, then 1)
C = {q2}	∅	D = {q3}	D = {q3}
… continues to the accept set			

Algorithm: (1) start state = ε-closure of q0; (2) for each unmarked DFA state S and symbol a, compute T = ε-closure(move(S,a)); (3) add T if new; (4) a DFA state accepting if it contains an NFA accepting state. Then **minimise** by partition refinement: start with {accepting}, {non-accepting}, split any block whose members go to different blocks on some symbol, repeat until stable. Report state counts before and after **[from your construction]**.

Equivalence with regular expressions

- **Thompson's construction (regex → NFA):** for `ab|c`, build a small NFA for each symbol, join a and b by an ε-transition (concatenation), then add a new start state with ε-edges to both branches and a new accept state (union). Star adds an ε-loop and ε-skip.
- **Automaton → regex (state elimination):** for the DFA accepting digits followed by optional "." digits, remove the middle state and combine edges: result `\d+(\.\d+)?`. The result matches the original machine, proving equivalence both ways.

Pumping lemma proof (non-regular language)

Lemma: if L is regular, there exists $p \geq 1$ such that any $s \in L$ with $|s| \geq p$ can be written $s = xyz$ with $|xy| \leq p$, $|y| \geq 1$, and $xy^iz \in L$ for all $i \geq 0$.

Claim: $L = \{ (^n)^n : n \geq 0 \}$ is not regular.

- Assume L is regular with pumping length p. Take $s = (^p)^p \in L$.
- Since $|xy| \leq p$, both x and y lie in the first p characters, so y consists only of "(" and $|y| \geq 1$.
- Then xy^2z has more "(" than ")", so $xy^2z \notin L$. Contradiction.

Therefore L is not regular, which confirms the Task 3 finding that balanced nested brackets in clinical notes cannot be matched by a regex or FSA.

Task 5 – Inflectional Morphology (3 marks)

Definitions

- **Morpheme:** the smallest meaning-bearing unit. **Stem:** the base to which affixes attach. **Affix:** a bound morpheme (prefix, suffix). **Inflection:** affixation that gives a grammatical form of the same word. **Paradigm:** the full set of inflected forms of a lexeme.

Inflection vs derivation

Criterion	Inflection	Derivation
Category	Preserved (fracture N → fractures N)	Often changed (operate V → operation N)
Productivity	Highly productive, near-regular	Less productive, many gaps
Position	Outermost (after derivational affixes)	Inside inflection

Paradigms in clinical English

Lexeme	Base	3SG / PL	Past / -ed	-ing / comparative
fracture (N/V)	fracture	fractures	fractured	fracturing
operate (V)	operate	operates	operated	operating
infection (N)	infection	infections	–	–
suture (V)	suture	sutures	sutured	suturing
severe (Adj)	severe	–	–	severer / severest

Paradigm types: noun number (-s), verb tense/aspect (-ed, -ing), subject agreement (-s), adjective comparison (-er, -est).

Orthographic rules with counter-examples

Rule	Statement	Over-generates on
E-deletion	Drop final e before -ing/-ed (make → making)	see + ing → *seing; dye + ing → *dying (needs the exception dyeing)
Consonant doubling	Double final consonant after a stressed CVC (stop → stopped)	visit + ed → *visitted (unstressed final syllable)
Y → I	Final consonant+y → ie/i before -s/-ed (study → studies)	play + s → *plaies (vowel before y)
Sibilant -es	Insert e after s, x, z, ch, sh before -s (box → boxes)	stomach + s: ch is pronounced /k/, so stomachs, not *stomaches

Irregular forms

They must be listed in the lexicon because no productive rule derives them: femur → femora, bacterium → bacteria, vertebra → vertebrae, (to) take → took, (to) go → went, (to) bleed → bled, (to) fall → fell.

Measurement

- **Type-to-lemma ratio** = distinct word forms ÷ distinct lemmas **[from your run]**.
- **Proportion of query failures explained by inflection alone** = failed queries fixed by lemmatising ÷ all failed queries **[from your run]**.

Task 6 – Derivational Morphology (4 marks)

Derivational affixes (at least 15)

Affix	Type	Category change	Example
-itis	suffix	N → N (inflammation)	arthritis
-ectomy	suffix	N → N (removal)	nephrectomy
-ology	suffix	N → N (study of)	cardiology
-osis	suffix	N → N (condition)	fibrosis
-aemia	suffix	N → N (blood condition)	anaemia
-ation	suffix	V → N	operation
-able	suffix	V → Adj	treatable
-ive	suffix	V → Adj	invasive
-ic	suffix	N → Adj	cardiac
-ment	suffix	V → N	treatment
-al	suffix	N → Adj	renal / medical
nephro-	prefix	N/Adj → N/Adj (kidney)	nephrotic
hyper-	prefix	no change (excess)	hypertension
hypo-	prefix	no change (deficit)	hypotension

Affix	Type	Category change	Example
re-	prefix	V → V (again)	re-admit
un-	prefix	Adj → Adj (not)	unstable

Derivational families and impossible orders

- **operate:** operate → operation → operational → operationally.
- **infect:** infect → infection → infectious → infectiously.
- **treat:** treat → treatment, treatable → untreatable.
- **nephr-:** nephro- + -ectomy → nephrectomy; nephro- + -itis → nephritis.

Impossible orders: *-ation + re- ("operationre"), since prefixes cannot follow suffixes; *-itis + nephro- ("itisnephro"), for the same reason; *un-treat-able-ment violates the category requirements (-ment needs a verb).

Non-compositionality

Meaning cannot always be predicted from the parts: "operation" can mean surgery or a mathematical or military operation; "hypertension" is not literally "above tension" in the everyday sense; "patient" (noun) and "patient" (adjective, calm) are unrelated. Aggressive stemming that merges these will return wrong records, so conflation must be conservative.

Productivity

Count types and tokens per affix in your corpus **[from your run]**. Classification rule: **productive** if the affix has many types and many hapaxes (words seen once), and it attaches to new roots; **fossilised** if it has few types, no hapaxes and appears only in fixed words.

Should derivational relatives be conflated?

	Helps the clinician	Misleads the clinician
Example 1	operate / operation (find all surgery records)	hypertension / hypotension (opposite conditions)
Example 2	fracture / fractured (same finding)	organ / organic / organism (different concepts)

Position: conflate inflection fully; conflate derivation only through a whitelist of safe pairs, never through blind affix stripping.

Task 7 – Finite-State Morphological Parsing (5 marks)

Finite-state transducer

An FST is a two-tape automaton whose arcs carry pairs a:b. The **lexical tape** holds stem plus features (fracture+N+PL); the **surface tape** holds the written form (fractures). Worked example:

Step	Lexical tape (read)	Surface tape (written)
1	f r a c t u r e	f r a c t u r e (identity arcs)
2	+N	ε
3	+PL	s (after ^ boundary)
Result	fracture+N+PL	fractures

Lexicon as continuation classes

Class	Members	Continues to
Reg-Noun	fracture, wound, infection	Noun-Infl, Deriv-Suffix
Reg-Verb	operate, treat, suture	Verb-Infl, Deriv-Suffix
Adjective	severe, acute, renal	Adj-Infl (-er, -est)
Irreg-Noun	femur, bacterium	femora, bacteria (listed forms)
Noun-Infl	+N +SG | +N +PL	end (ε or s)
Verb-Infl	+V +3SG | +PAST | +PRESPART	end (s, ed, ing)
Deriv-Suffix	-ation, -ment, -able, -ive	Noun-Infl / Adj-Infl

Hand traces (five forms)

Surface	State sequence	Analysis
fractures	q0 → Reg-Noun (f-r-a-c-t-u-r-e) → Noun-Infl (s ↔ +PL) → end	fracture +N +PL
operating	q0 → Reg-Verb (operat) → e deleted by rule → Verb-Infl (ing ↔ +PRESPART)	operate +V +PRESPART
treatment	q0 → Reg-Verb (treat) → Deriv-Suffix (ment ↔ ^DER-MENT) → Noun-Infl (ε ↔ +SG)	treat +V ^MENT +N +SG
bacteria	q0 → Irreg-Noun (bacteria ↔ bacterium +N +PL)	bacterium +N +PL
nephrectomies	q0 → nephro- → ectomy → Noun-Infl (ies ↔ y +PL)	nephrectomy +N +PL

Generation direction

Swap the roles of the two tapes: give the machine fracture+N+PL as input and read fractures as output. Parsing and generation use **one machine** because an FST is a relation between the tapes and is not directional. Only the reading direction changes.

Ambiguity

Surface	Analyses returned (all)	Chosen downstream by
leaves	leaf +N +PL; leave +V +3SG	POS tagger / context
discharged	discharge +V +PAST; discharge +V +PASTPART; discharged +Adj	POS tagger
found	find +V +PAST; found +V +BASE	POS tagger

The transducer returns all analyses as a set. A downstream part-of-speech tagger or a language model (probabilistic component) scores them using sentence context and keeps the most likely one. For search, one may simply index all lemmas.

Task 8 – Orthographic Rules and Two-Level Morphology (4 marks)

Why concatenation fails

Stem + suffix	Plain concatenation	Correct surface
fox + s	foxs	foxes
make + ing	makeing	making
study + s	studys	studies
stop + ed	stoped	stopped

Orthographic rules (rewrite notation, ^ = morpheme boundary, # = word end)

Rule	Notation	Context explained
E-deletion	e → ε / C __ ^ (ing | ed)	delete e after a consonant before -ing/-ed
E-insertion	ε → e / (x | s | z | ch | sh) ^ __ s	insert e between sibilant and -s
Y-replacement	y → ie / C ^ __ s	change y to ie after a consonant before -s
Consonant doubling	ε → C_1 / V C_1 ^ __ (ing | ed)	double the final consonant after a stressed CVC
Boundary deletion	^ → ε / __ #	remove the boundary marker at the end

Cascade

Each rule is compiled to an FST. Compose them with the lexicon: Lexicon ∘ $Rule_1$ ∘ $Rule_2$ ∘ … ∘ $Rule_5$. **Order dependencies:** consonant doubling and e-deletion must apply before boundary deletion, since they refer to ^. Y-replacement must precede e-insertion, so that studies is not re-processed.

```
lexical tape -> [Lexicon FST] -> intermediate (fox^s) -> [E-insertion] -> fox^es -> [Boundary del.] -> foxes
```

Cascade vs two-level (Koskenniemi)

	Sequential cascade	Parallel two-level
Operation	Rules applied one after another	All rules run simultaneously as constraints on lexical:surface pairs
Advantage	Easy to write and reason about step by step	No intermediate strings; no rule-ordering problems
Disadvantage	Order matters; intermediate forms needed	Rules are harder to write as constraints

Testing

Test at least 20 forms exercising every rule (for example boxes, buses, watches, making, hoped, stopped, running, studies, tries, plays, fractured, sutured). Tabulate expected vs produced and account for every failure **[from your run]**. Typical failure causes: irregular forms (ran, went), unstressed doubling (visited), and foreign plurals.

Task 9 – The Porter Stemmer (5 marks)

Definitions

- A **consonant** is any letter other than a, e, i, o, u, and other than y when preceded by a consonant. Otherwise it is a vowel.
- Every word has the form [C](VC)^{m}[V], and **m** is the number of VC sequences.
- ***v\ the stem contains a vowel.** *d: **the stem ends with a double consonant.** *o:** the stem ends consonant-vowel-consonant where the last consonant is not w, x or y.

Measure examples

Word	C/V form	m	Calculation
tree	C V (tr-ee)	0	no VC pair
trouble	C V C V (tr-ou-bl-e)	1	one VC: ou-bl
fracture	C V C V C V (fr-a-ct-u-r-e)	2	VC: a-ct, u-r
infection	V C V C V C (i-nf-e-ct-io-n)	3	VC: i-nf, e-ct, io-n
sutures	C V C V C (s-u-t-u-r-es)	2	VC: u-t, u-r-es
renal	C V C V C (r-e-n-a-l)	2	VC: e-n, a-l

Algorithm steps and traces

Steps: 1a plurals (sses→ss, ies→i, s→ε); 1b -ed/-ing (if *v*); 1c y→i (if *v*); 2 double suffixes (-ational→-ate, etc.); 3 (-icate→-ic, etc.); 4 remove suffixes if m>1; 5a drop final e (if m>1, or m=1 and not *o); 5b -ll→-l (if m>1).

Word	Rule firing sequence	Final stem
fractures	1a s→ε: fracture; 5a m=2>1 drop e: fractur	fractur
fractured	1b ed→ε (stem has vowel): fractur	fractur
fracturing	1b ing→ε: fractur	fractur
infections	1a: infection; 2 –; 3 –; 4 -ion (m>1, preceded by t): infect	infect
operational	2 -ational→-ate: operate; 4 -ate (m>1): oper; 5a –	oper
nephritis	1a (s→ε not applied: ends -is); no later rule matches	nephriti

Stemming vs lemmatisation

Run the same 200 annotated forms through Porter and through the Task 7 transducer and tabulate agreement. Typical divergence:

Form	Porter	FST lemma	Comment
fractures	fractur	fracture	agree on grouping, differ in form
bacteria	bacteria	bacterium	Porter misses irregular plural
analyses	analys	analysis	Porter splits from analysis → analysi
operation	oper	operation	Porter over-reduces

Error types

- **Over-stemming** (words that must stay distinct are merged): organization / organ → organ; operate / operation / operator → oper; general / generous.
- **Under-stemming** (related words not merged): analysis → analysi but analyses → analys; bacterium vs bacteria.

Rate of each = erroneous merges (or missed merges) ÷ total word pairs in the 200-form annotated set **[from your run]**.

Retrieval experiment

Run the 40 queries under three conditions and report P, R and F1 **[from your run]**.

Condition	Precision	Recall	F1	Expected trend
No stemming	[ ]	[ ]	[ ]	highest P, lowest R
Porter	[ ]	[ ]	[ ]	highest R, lowest P
Full morphological analysis	[ ]	[ ]	[ ]	best balance (F1)

Task 10 – Implement and Test the Pipeline (3 marks)

Implementation outline (standard library + NLTK only)

```
import re, time
from nltk.stem import PorterStemmer

TOKEN = re.compile(r'(?:[A-Za-z]\.){2,}|\d+(?:\.\d+)?|\w+(?:-\w+)*|[^\w\s]')

class NFA:
    def __init__(self, delta, start, accept): ...      # delta: {(state, sym): set(states)}
    def eps_closure(self, S): ...
    def accepts(self, s): ...                          # simulate with live-state sets
    def to_dfa(self): ...                              # subset construction

class FST:
    def analyse(self, surface): ...                    # returns all (stem, features)
    def generate(self, stem, feats): ...               # same arcs, tapes swapped

stemmer = PorterStemmer()
```

Run report (table per stage)

Stage	Processed	Matched/analysed	Rejected	Run-time
Tokenizer	[ ]	[ ]	[ ]	[ ]
Entity regex	[ ]	[ ]	[ ]	[ ]
Automata validators	[ ]	[ ]	[ ]	[ ]

Stage	Processed	Matched/analysed	Rejected	Run-time
FST analyser	[ ]	[ ]	[ ]	[ ]
Porter fallback	[ ]	[ ]	[ ]	[ ]

Test cases

- At least one accepted and one rejected string per automaton and per regex.
- Ambiguous forms from Task 7 (leaves, discharged, found) must return all analyses.
- Irregulars from Task 5 (femora, bacteria, vertebrae, took) must resolve to the right lemma.
- Compare the code output with your hand-computed results from Tasks 4, 7 and 9.

Plots

At least one plot, for example analyser coverage (%) against lexicon size, or run-time against record length.

Task 11 – Compare, Recommend and Defend (2 marks)

Criterion	No normalisation	Porter stemming	Full FST analysis
Retrieval quality	High P, low R	High R, lower P	Best F1
Coverage	Exact words only	All words (no lexicon)	Only in-lexicon words
Run-time	Fastest	Fast	Slowest of the three
Maintenance effort	None	Almost none	High (lexicon, rules, annotation)

Recommendation: FST analyser as the main component, with Porter as fallback for out-of-lexicon words, and a conservative whitelist for derivational conflation.

- **Expected accuracy:** [from your measured F1].
- **Failure modes:** unknown drug names or new terms not in lexicon; wrong choice among ambiguous analyses; abbreviation ambiguity.
- **What a clinician would notice:** a search for "fracture" misses a record that uses a rare spelling, or returns unrelated "organ" records for "organic" (if Porter over-stems).
- **Effort and economics:** lexicon building, 15+ affixes, pattern writing and annotation of 200+ forms and 40 queries all take expert hours. A learned approach becomes more economical once the vocabulary, noise and language variety grow beyond what can be hand-annotated, and when a large amount of labelled data is available.

Task 12 – Day-4 Design Challenge (8 marks)

A. Impact Analysis (2 marks)

New input property	What breaks	Reason
Character noise (0/O, 1/l, rn/m)	Entity regexes and automata	Exact match accepts or rejects; 5Omg is rejected
Hyphenation across lines	Tokenizer, analyser	One word becomes two fragments (frac- ture)
Lost diacritics	Regional-language stems	Lexicon entries no longer match

New input property	What breaks	Reason
Out-of-lexicon stems	FST analyser	Closed vocabulary assumption fails
Non-English morphology	Affix rules	Rules are English-only (-irundhuchu has no English analysis)
Code-mixing	Whole pipeline	Language changes within a sentence

Assumptions broken: clean typed input, closed vocabulary, English-only morphology, and exact accept/reject matching.

Estimated effect: if at least 20% of tokens are noisy or unknown, analyser coverage drops by roughly that share (e.g. clean coverage 90% → about 72%). Pattern recall drops most for digit-heavy entities (dose, dates, phone) because one confused character rejects the whole entity. Precision falls only slightly since noise rarely creates false matches. Use your Task 2 and Task 7 error rates as the baseline **[from your run]**.

B. Evaluation of the Original Design (2 marks)

Record	Input	What fails first	Why
OCR	"Paracetamol 5OOmg bd, pt c/o fr-\nacture"	Strength regex (5OOmg)	O is not in \d
Code-mixed	"patient ku fever irundhuchu, paracetamol 500mg kuduthen"	Analyser on "irundhuchu", "kuduthen"	Not in English lexicon; no analysis returned
Both	"Paracetarnol 5OOmg kuduthen, f1ever"	Tokeniser/regex first, then analyser	rn for m, O for 0, 1 for i, non-English verb

Degradation: report pattern precision/recall, analyser coverage and retrieval F1 on a noisy sample and compare with the clean baseline **[from your run]**.

Alternative formalisms: a weighted edit-distance transducer (OCR confusions cost less than random substitutions) and a character n-gram language identifier are cheaper than extending the rule set for every confusion and every regional word.

C. Adaptation or Redesign (2 marks)

Decision: MODIFY (keep the finite-state core, supplement it).

Component	Change
Pre-processing	Add de-hyphenation across line breaks (join word- + newline + continuation)
Patterns	Loosen character classes: [0Oo], [1lI], and allow rn/m variants in unit and digit contexts
Automata	Add weighted edit-distance transducer (OCR-confusion costs) before the analyser; accept strings within a cost threshold
Language ID	Add a character n-gram classifier per token: English tokens go to the FST, Romanised regional tokens go to a small regional lexicon or Porter fallback
Lexicon	Add regional stems (e.g. kudu, iru) and clinical loanwords
Rule cascade	Unchanged for English; fallback to Porter or character n-gram matching for out-of-lexicon tokens

Effect (estimate, replace with measurements): coverage rises towards the clean level, recall improves, precision falls slightly because fuzzy matches add false hits, run-time increases (edit-

distance search), and annotation effort increases for OCR-confusion weights and regional lexicon entries.

D. Evidence-Based Defence (2 marks)

- **Pattern error analysis (Task 2):** most clean-corpus errors were format variants, which loosened classes and fuzzy matching address directly.
- **Automaton traces (Task 4):** exact accept/reject is the root cause of OCR failure, which justifies a weighted front end rather than a rewrite.
- **Coverage and retrieval figures (Tasks 7 and 9):** FST with Porter fallback gave the best F1 on clean data, so the working resources are retained and extended.
- **Effort estimates (Task 11):** a full redesign would discard resources already built; a modification costs less.

Remaining limitations: a finite-state layer will never resolve context-dependent meaning, nested structure (not regular), or true semantic ambiguity (e.g. which meaning of MS applies), and it cannot learn new regional vocabulary by itself, however much it is extended. These need a statistical or neural component, which is the natural next step once enough annotated data exists.